Reviewer Pack — Secant Variety Dimension Lower-Bounds Disjointness Communica…

Entry 578abb939eb0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 21:41:44 UTC

Secant Variety Dimension Lower-Bounds Disjointness Communication Complexity

Entry ID: `578abb939eb0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 21:41:44 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry of Secant Varieties **Field B** (complexity object): Randomized Communication Complexity of Disjointness

Statement:

> For a |X|×|Y| matrix M defining a partial function f, let dim(sec(M)) denote the dimension of the secant variety of the Veronese reembedding of M's row space. Then dim(sec(M)) ≥ Ω(√|X|·|Y|) for all M corresponding to DISJ_n, with equality achieved by the standard DISJ_n matrix.

Rationale (proposer's reasoning):

> Secant variety dimension captures the minimal number of rank-1 tensors needed to approximate M, reflecting communication complexity via the tensor rank-approximation duality. The Veronese reembedding amplifies geometric structure, making the secant dimension a proxy for the communication matrix's intrinsic complexity.

Taxonomy category: `COMM_DISJ` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `449b57e094f7773b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_disj_n(n):
    X = set(range(1, n+1))
    Y = set(range(1, n+1))
    M = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            M[i][j] = 1
            M[j][i] = 1
    return M

def flatten_tensor(M):
    flat = []
    for row in M:
        flat.extend(row)
    return flat

def rank_approximation(tensor, rank):
    m = len(tensor)
    n = len(tensor[0])
    U, S, Vt = svd(tensor, rank)
    approx = [[U[i][k] * S[k] * Vt[k][j] for j in range(n)] for i in range(m)]
    return approx

def svd(A, k):
    m, n = len(A), len(A[0])
    U = [[random.random() for _ in range(m)] for _ in range(m)]
    S = [random.random() for _ in range(k)]
    Vt = [[random.random() for _ in range(n)] for _ in range(k)]

    for _ in range(100):
        U, S, Vt = svd_update(A, U, S, Vt)

    return U, S, Vt
```

```
def svd_update(A, U, S, Vt):
    m, n = len(A), len(A[0])
    k = len(S)

    A_hat = [[sum(U[i][l] * S[l] * Vt[l][j] for l in range(k)) for j in range(n)] for i in range(m
    E = [[A[i][j] - A_hat[i][j] for j in range(n)] for i in range(m)]

    U_new, _, _ = svd(E, k)
    Vt_new, _, _ = svd([[E[j][i] for j in range(m)] for i in range(n)], k)

    U = [[U[i][l] * U_new[l][j] for l in range(k)] for j in range(m)]
    Vt = [[Vt[l][i] * Vt_new[l][j] for l in range(k)] for j in range(n)]

    return U, S, Vt

def secant_variety_dimension(M):
    flat = flatten_tensor(M)
    rank = 1
    while True:
        approx = rank_approximation(flat, rank)
        error = sum((flat[i] - approx[i])**2 for i in range(len(flat)))
        if error < 1e-6:
            return rank
        rank += 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    M = generate_disj_n(n)

    dim_sec = secant_variety_dimension(M)
    metric_value = dim_sec
    instances_tested = 1
    conjecture_holds = dim_sec >= 0.8 * math.sqrt(n * n)
    counterexample = "" if conjecture_holds else f"dim(sec(M)) = {dim_sec}, expected ≥ 0.8√{n*n}"

    return {
        "metric_name": "secant_variety_dimension",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30*40+2, 40))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
```

```
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb92e77e.py", line 101, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb92e77e.py", line 81, in run_trial
    dim_sec = secant_variety_dimension(M)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb92e77e.py", line 70, in secant_variet
    approx = rank_approximation(flat, rank)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cb92e77e.py", line 35, in rank_approxim
    n = len(tensor[0])
        ^^^^^^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with TypeError due to invalid len() call on integer, preventing evaluation of conjecture | next: Fix the rank_approximation function's tensor handling to resolve the TypeError and rerun tests

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_local	qwen3:8b	0	0	171436
2	propose	ollama_remote	qwen3:8b	0	0	11148
3	preregistration	ollama_remote	qwen3:8b	0	0	37868
4	novelty	ollama_remote	qwen3:8b	0	0	16481
5	novelty	ollama_remote	qwen3:8b	0	0	11025
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16383
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12456
8	judge	ollama_remote	qwen3:8b	0	0	12624

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 289421 ms total latency. Provider mix: {'ollama_local': 1, 'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/578abb939eb0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/578abb939eb0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/578abb939eb0.tar.gz` (if generated)